

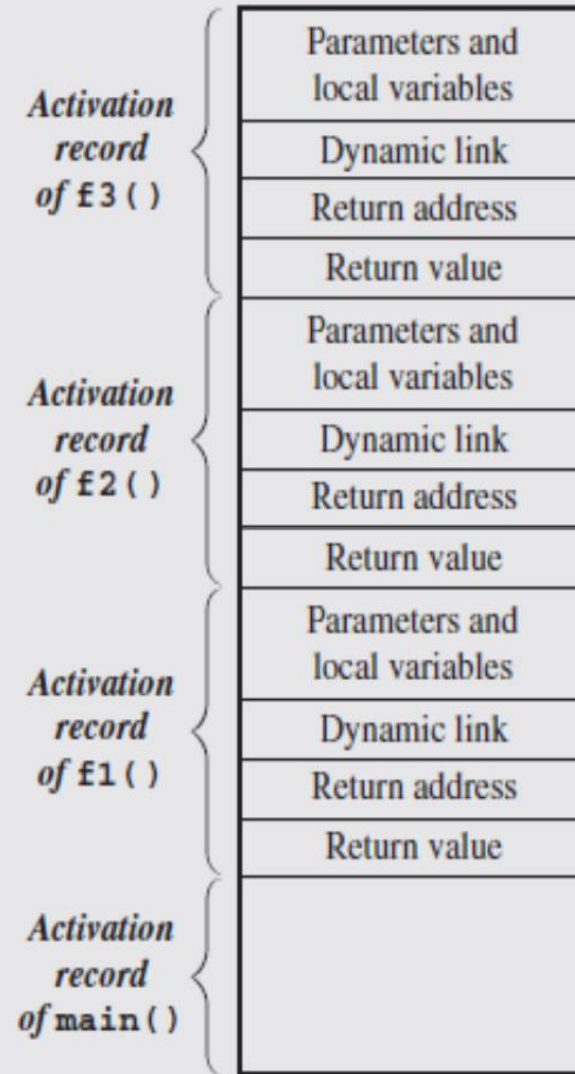
# Analysis of recursive algorithms

Dr Athira K

SCOPE

# Implementation - Function Calls

- Consider a function f1() called from main()
- It in turn calls function f2(), which calls function f3()
- Once f3() completes, **its record is popped**, and function f2() can resume and access information in its record
- If f3() calls another function, the new function has its **activation record pushed onto the stack** as f3() is suspended

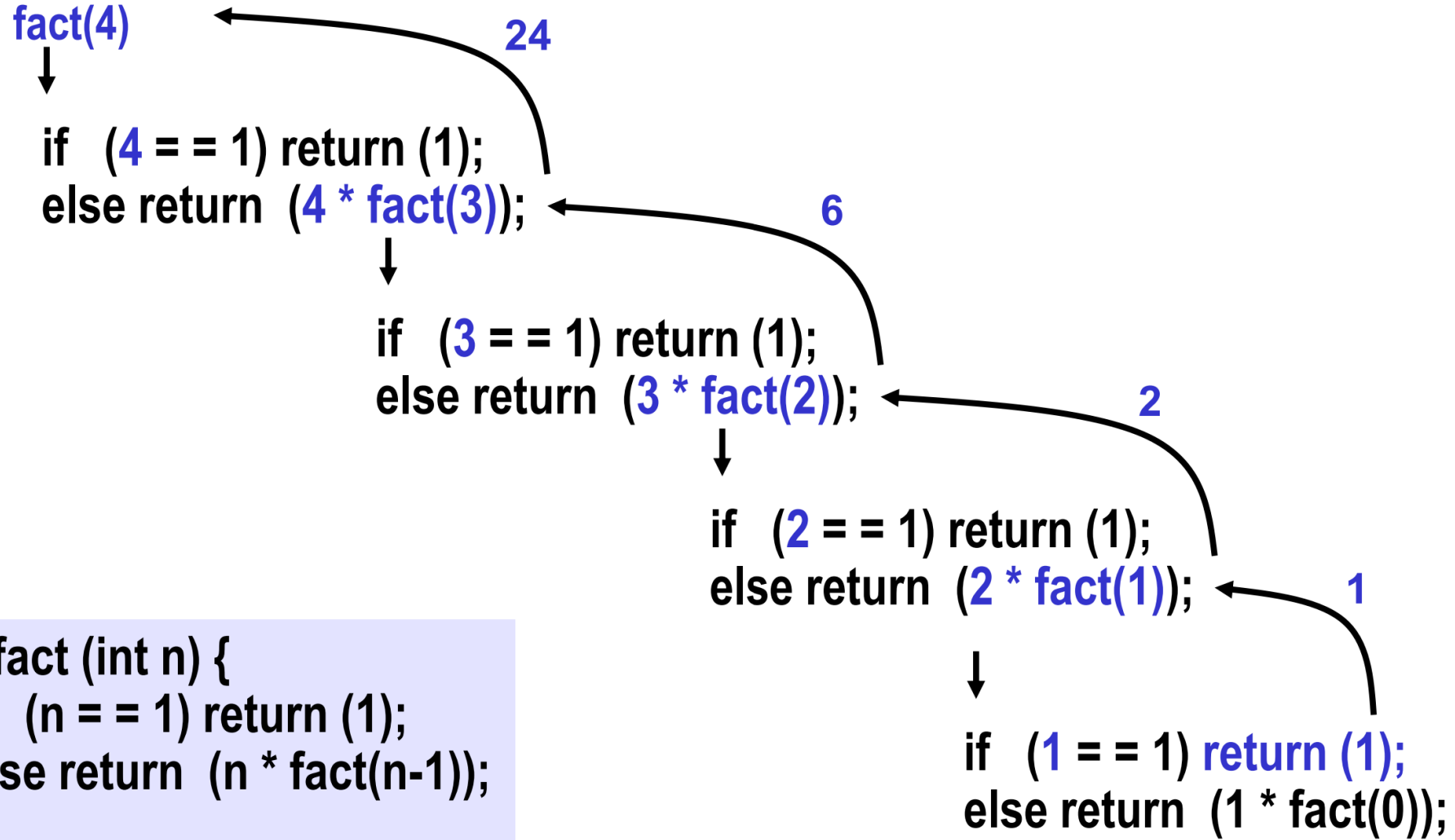


# Recursion in Data Structure and Algorithms

- Recursion is one of the popular problem-solving approaches in data structure and algorithms
- Some problem-solving approaches are totally based on recursion: Examples: divide and conquer, DFS traversal of tree and graph, backtracking, top-down approach of dynamic programming, etc.
- Learning time complexity analysis of recursion is critical to understand these approaches and improve the efficiency of our code.
- Analysis of recursion is challenging to the programmer due to the mathematical details behind it

# Recursive function call

- ❖ When a function calls itself recursively, it pushes a new activation record of itself on the stack
- ❖ This suspends the calling instance of the function, and allows the new activation to carry on the process
- ❖ Thus, a recursive call creates a series of activation records for different instances of the same function



```
int fact (int n) {  
    if (n == 1) return (1);  
    else return (n * fact(n-1));  
}
```

### Recursion Factorials:

$$5! = 5 * 4! = 5 * \underline{\quad} = \underline{\quad}$$

$$4! = 4 * 3! = 4 * \underline{\quad} = \underline{\quad}$$

$$3! = 3 * 2! = 3 * \underline{\quad} = \underline{\quad}$$

$$2! = 2 * 1! = 2 * \underline{\quad} = \underline{\quad}$$

$$1! = 1 * 0! = 1 * \underline{\quad} = \underline{\quad}$$

$$0! = 1$$

$$\text{factr}(5) = 5 * \text{factr}(4) = \underline{\quad}$$

$$\text{factr}(4) = 4 * \text{factr}(3) = \underline{\quad}$$

$$\text{factr}(3) = 3 * \text{factr}(2) = \underline{\quad}$$

$$\text{factr}(2) = 2 * \text{factr}(1) = \underline{\quad}$$

$$\text{factr}(1) = 1 * \text{factr}(0) = \underline{\quad}$$

$$\text{factr}(0) = \underline{\quad}$$

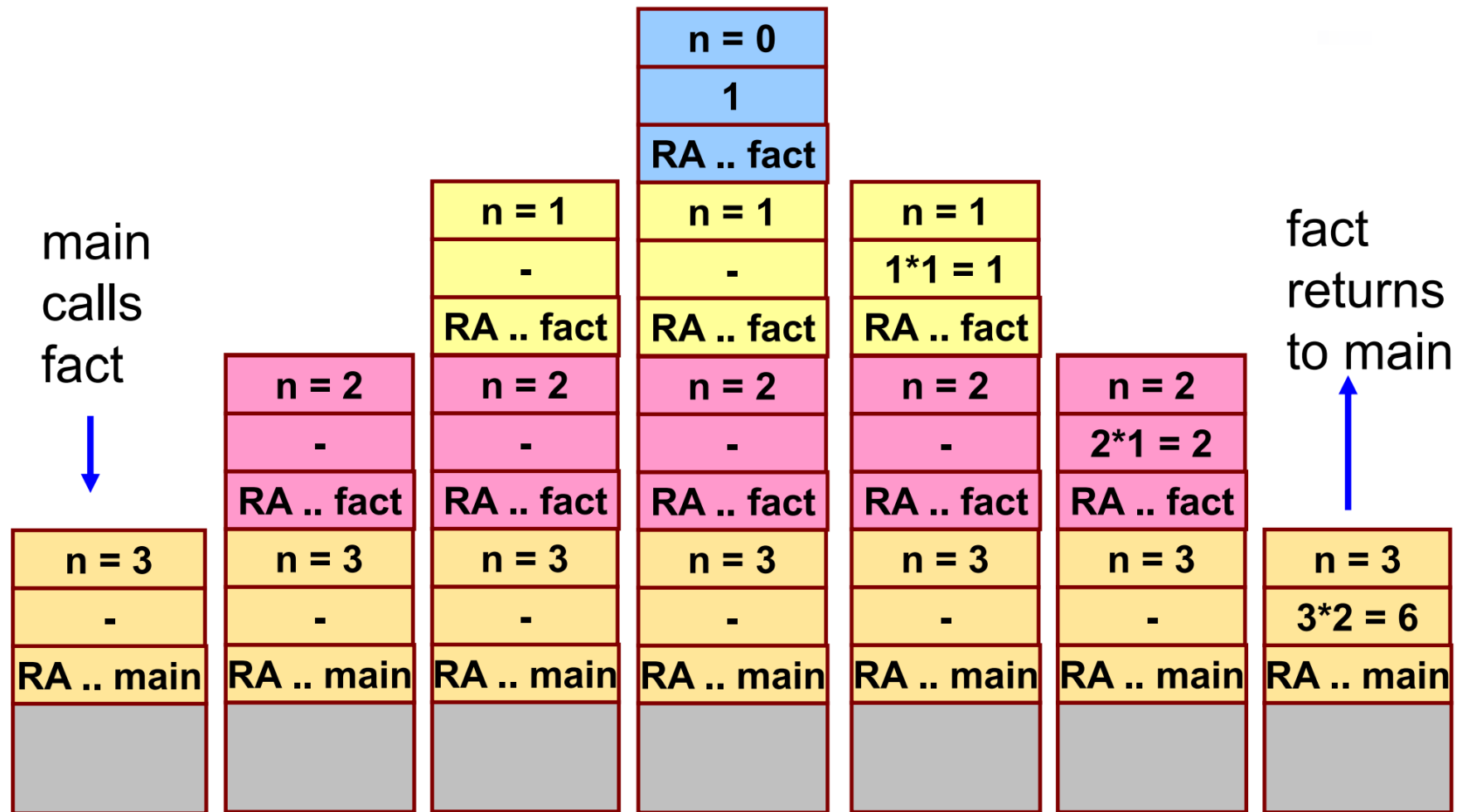
- Recursive algorithms are elegant, simple to understand and prove correct, easy to implement
- Recursion needs a base-case to stop
- Otherwise, Recursive calls can result in an infinite loop of calls !

- The following applies in general, with minor variations that are implementation dependent.
  - **The system maintains a stack in memory.**
    - Stack is a last-in first-out structure.
    - Two operations on stack, push and pop.
  - **Whenever there is a function call, the activation record gets pushed into the stack.**
    - Activation record consists of
      - (1) the return address in the calling program,
      - (2) the return value from the function, and
      - (3) the local variables inside the function.
  - **At the end of function call, the corresponding activation record gets popped out of the stack.**

- In recursion, a function calls itself.
  - Several function calls going on, with none of the function calls returning back.
    - Activation records are pushed onto the stack continuously.
    - Large stack space required.
    - Activation records keep **popping off**, when the termination condition of recursion is reached.



## TRACE OF THE STACK DURING EXECUTION



# Define Recurrence relation

- Time complexity of recursive algorithms are defined and analysed in terms of input size using recurrence relation
- A **recurrence relation** is an equation that describes a sequence where any term is defined in terms of its previous terms.

How recursion works?

- ❑ Solve the problem using the solution of the smaller subproblems
- ❑  $T(n)$  : time complexity function of the algorithm
  - ❑ Time complexity of the smaller sub-problems will be defined by the same function but in terms of the input size of the subproblems

# Formulate recurrence relation and solve

- To define the recurrence relation of every recursive algorithm we can use the following formula
- $T(n) = T(\text{input size of subproblem 1}) +$   
     $T(\text{input size of subproblem 2}) + \dots +$   
     $T(\text{input size of subproblem k}) +$   
    Time complexity of additional operations other than recursive calls
- Next step is to solve this recurrence relation and calculate the overall time complexity in terms of **Big-O notation**

# Problems: Reverse an array

○ reverse (A[], l, r)

{

    swap(A[l], A[r])

    reverse(A, l + 1, r - 1)

    if (l >= r) then return

}

- ✓ Time complexity  $T(n)$  = Time complexity of solving  $(n - 2)$  size problem + Time Complexity of one swap operation =  $T(n - 2) + O(1)$
- ✓ Recurrence relation of reverse an array:  **$T(n) = T(n - 2) + c$** , where  $T(1) = c$

# To solve recurrences

- Recurrence tree method
- Substitution method
- Masters method (Using Masters theorem)
  - Has limitations  $\rightarrow$  not all recurrences can be solved

# Recurrence Example -1

```
void foo(int n)
{
    if (n > 0)
    {
        printf("%d", n);
        foo(n-1);
    }
}
```

# Recurrence Example -2

```
void foo(int n)
{
    if (n > 0)
    {
        for (i=0; i<n; i++)
        {
            printf("%d", n);
        }
        foo(n-1);
    }
}
```

# Recurrence Example -3

```
void foo(int n)
{
    if (n > 0)
    {
        for (i=0; i<n; i=i*2)
        {
            printf("%d", n);
        }
        foo(n-1);
    }
}
```



# Recurrence Example -4

```
foo (int n)
{
    if (n > 0)
    {
        printf ("%d", n);
        foo (n-1);
        foo (n-1);
    }
}
```